A Hybrid EDHOC Protocol Post-Quantum Secure against Key Compromise Impersonation*

SeongHan Shin

National Institute of Advanced Industrial Science and Technology (AIST)
2-3-26 Aomi, Koto-ku, Tokyo, 135-0064 Japan
seonghan.shin@aist.go.jp

Abstract

Recently, the IETF LAKE working group has standardized the EDHOC (Ephemeral Diffie-Hellman Over COSE) protocol [52] that is a lightweight and application-layer authenticated key exchange protocol tailored for resource-constrained IoT devices and networks. In this paper, we propose a hybrid EDHOC (for short, Hybrid-EDHOC2) protocol that provides both classical security and PQ security by instantiating KEM (resp., signature scheme) with PQ KEM (resp., PQ signature scheme). In the Hybrid-EDHOC2 protocol, initiator I and responder R share an authenticated session key. Also, we discuss several classical and PQ security properties (mutual authentication, security of session key, forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation) of the Hybrid-EDHOC2 protocol. In addition, we conduct extensive comparisons of the EDHOC [52], PQ-EDHOC [31], PQ-EDHOC1 [30], PQ-EDHOC2 [30], Hybrid-EDHOC [66], and Hybrid-EDHOC2 protocols with respect to security properties and computation/communication efficiency.

1 Introduction

With the rapid and widespread deployments of the IoT (Internet of Things), many cryptographic protocols have been studied to meet several constraints (e.g., low processing power, small memory, limited bandwidth, and battery lifetime) in the literature. An authenticated key exchange protocol is a core cryptographic primitive that allows communicating peers to authenticate each other and share a session key. Recently, the IETF (Internet Engineering Task Force) LAKE (Lightweight Authenticated Key Exchange) working group [10] has standardized the EDHOC (Ephemeral Diffie-Hellman Over COSE) protocol [52, 53] that is a lightweight and application-layer authenticated key exchange protocol tailored for resource-constrained IoT devices and networks (e.g., [41, 45]). According to [27], EDHOC achieves maximum 14 times reduction in message size, 1.44 times improvement in handshake duration, and 2.79 times reduction in energy consumption compared to DTLS 1.3 [50]. On the other hand, the long-term operation of IoT devices introduces a new challenge for classical cryptographic protocols, as Shor's algorithm [67] can efficiently solve the integer factorization and (finite field or elliptic curve) discrete logarithm problems with large-scale quantum computers. In response to the potential future threat, the NIST (National Institute of Standards and Technology) has initiated a PQC (Post-Quantum Cryptography) project [13] to standardize PQ (Post-Quantum) cryptographic algorithms. These developments underscore the urgency of transitioning Internet security protocols to PQ-secure ones.

The EDHOC protocol is designed following a SIGMA family [59] of key exchange protocols that introduce a general approach to authenticated Diffie-Hellman protocols using a combination of signature and MAC (Message Authentication Code). SIGMA is widely used and is a cryptographic basis of many

*Proceedings of The 2026 IFIP WG 8.4 International Symposium on E-Business Information Systems Evolution (EBISION 2026), Article No. 48, July 8-10, 2026, Seoul, South Korea.
© The copyright of this paper remains with the author(s).

security protocols (e.g., IKEv2 [48] and TLS 1.3 [49]). In order to reduce message sizes between the two peers (initiator and responder), SIGMA introduces a MAC-then-Sign variant where MAC is included in a signature message and MAC itself is not sent. Actually, the EDHOC protocol follows the MAC-then-Sign variant of the SIGMA-I protocol that protects the identity of the initiator. In addition to signature keys used for authentication, the EDHOC protocol allows static DH (Diffie-Hellman) keys (cf. EDHOC Authentication Method Type [52]) where a signature is replaced with a MAC, and a MAC key is an ephemeral-static DH key computed in a key schedule similar to the Noise XX pattern [12]. The use of static DH keys results in a significant reduction in message size compared to the use of signature keys.

There are some related works on the EDHOC protocol in the literature. In [65], E. L. Pérez et al. summarized formal symbolic/computational analysis results (e.g., [58, 64, 23, 32, 56, 28]) of earlier versions of the EDHOC protocol, and described how these vulnerabilities found in the security analyses were resolved to meet several security goals. In [31], L. P. Fraile et al. proposed a PQ version of the EDHOC (PQ-EDHOC) protocol by integrating PQ KEMs (Key Encapsulation Mechanisms) and PQ signature schemes, and evaluated various PQ KEM and PQ signature scheme combinations with respect to execution time, energy consumption, memory usage, and network performance. Their evaluation results indicate that ML-KEM [5, 60, 21, 55] emerges as the only suitable PQ KEM choice for resource-constrained environments, while other PQ signature schemes (e.g., HAWK [25]) show significant performance improvement over ML-DSA [5, 61] and FALCON [6] (to be standardized as FIPS (Federal Information Processing Standard)). Note that the PQ-EDHOC protocol is based on EDHOC Authentication Method Type 0 (Signature Key) [52]. Very recently, L. P. Fraile et al. [30] proposed two PQ variants of the PQ-EDHOC protocol (which we call PQ-EDHOC1 and PQ-EDHOC2) that use only a PQ KEM without a PQ signature scheme. However, their proposed protocols adopting an approach of PQNoise [19] require substantial modifications to the original EDHOC protocol [52, 53]. Specifically, the PQ-EDHOC1 protocol (Section III.B in [30]) needs additional fourth and fifth messages to be exchanged, and the PQ-EDHOC2 protocol (Section III.C in [30]) needs to send a KEM ciphertext, computed with the responder's KEM public key, in the first message.

1.1 Motivation and Our Contributions

For a smooth migration to PQC, international organizations [11, 1, 54, 36] recommend a hybrid scheme that combines a classical cryptographic algorithm/protocol with a PQ-secure one to guarantee security against both classical and quantum attackers. For example, ANSSI [1] emphasized that hybridation is crucial to cryptographic security and will remain mandatory until a complete transition to PQC. Up to now, many hybrid schemes (e.g., [20, 26, 51, 29, 35, 34, 43, 33, 44, 42, 39, 37, 38]) that provide both classical and PQ security have been studied and proposed in the literature and by international standard bodies. Very recently, Shin et al. [66] proposed the first hybrid EDHOC (Hybrid-EDHOC) protocol for both classical and PQ security, in an elaborate manner. However, the Hybrid-EDHOC protocol does not provide PQ security against key compromise impersonation.

In this paper, we propose another hybrid EDHOC (for short, Hybrid-EDHOC2) protocol that provides both classical security and PQ security by instantiating KEM (resp., signature scheme) with PQ KEM (resp., PQ signature scheme). Specifically, the Hybrid-EDHOC2 protocol is based on an authentication method of static ECDH keys and static PQ signature keys. In the Hybrid-EDHOC2 protocol, initiator I and responder R share an authenticated session key PRK_{out}. Unlike Hybrid-EDHOC [66], the Hybrid-EDHOC2 protocol provides double authentication based on a MAC and a PQ signature, and PQ security against key compromise impersonation. Also, we discuss several classical and PQ security properties (mutual authentication, security of session key, forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation) of the Hybrid-EDHOC2 protocol. In addition, we conduct extensive comparisons of the EDHOC [52], PQ-EDHOC

[31], PQ-EDHOC1 [30], PQ-EDHOC2 [30], Hybrid-EDHOC [66], and Hybrid-EDHOC2 (Section 3) protocols with respect to security properties and computation/communication efficiency.

2 Preliminaries

Table 1 shows abbreviations and notations to be used throughout this paper.

Table 1: Abbreviations and notations

	Meanings
I	Initiator identity
R	Responder identity
(a,A)	Static ECDH private-public key pair of I
(b,B)	Static ECDH private-public key pair of R
(sk_I,VK_I)	Static PQ signature signing-verification key pair of I
(sk_R,VK_R)	Static PQ signature signing-verification key pair of R
(x,X)	Ephemeral ECDH private-public key pair of I
(y,Y)	Ephemeral ECDH private-public key pair of R
(sk_{KEM},PK_{KEM})	Ephemeral PQ KEM private-public key pair of I
H	Cryptographic hash function
KDF	Key derivation function (EDHOC_KDF [52])
Extract	Key extraction function (EDHOC_Extract [52])
Expand	Key expansion function (EDHOC_Expand [52])
Enc	Symmetric-key encryption (XOR stream cipher or AEAD)
Dec	Symmetric-key decryption (XOR stream cipher or AEAD)
TH	Transcript hash
PRK	Pseudo-random key
EK	Encryption key
IV	Initialization vector
MK	MAC key
METHOD	EDHOC Authentication Method Type [52]
SUITES	EDHOC Cipher Suites [52] (an ordered set of algorithms for crypto agility, flexibility and modularity)
C	Connection Identifier
EAD	External Authorization Data
len	MAC length

2.1 Key Encapsulation Mechanism

A KEM (Key Encapsulation Mechanism) [24, 63] is a triple of the following algorithms.

- **(PQ-)KEM.KeyGen**(pp_{KEM}): On input of a public parameter pp_{KEM}, the key generation algorithm outputs a KEM private-public key pair (sk_{KEM}, PK_{KEM}).
- **(PQ-)KEM.Encaps**(PK_{KEM}): On input of a KEM public key PK_{KEM}, the encapsulation algorithm outputs a KEM shared key k_{KEM} and a KEM ciphertext C_{KEM}.
- **(PQ-)KEM.Decaps**(sk_{KEM}, C_{KEM}): On input of a KEM private key sk_{KEM} and a KEM ciphertext C_{KEM}, the decapsulation algorithm outputs a KEM shared key k_{KEM}.

Representative PQ KEMs include ML-KEM (CRYSTALS-Kyber) [5, 60, 21, 55], FrodoKEM [7, 21, 55], Classic McEliece KEM [4, 21, 55], and BIKE [3].

2.2 Signature Scheme

A signature scheme [57] is a triple of the following algorithms.

- **(PQ-)Sig.KeyGen**(pp_{Sig}): On input of a public parameter pp_{Sig}, the key generation algorithm outputs a signature signing-verification key pair (sk_{ID}, VK_{ID}).
- **(PQ-)Sig.Sign**(sk_{ID}, msg): On input of a signing key sk_{ID} and a message msg, the signing algorithm outputs a signature σ_{ID}.
- **(PQ-)Sig.Verify**$(VK_{ID}, msg, \sigma_{ID})$: On input of a verification key VK_{ID}, a message msg, and a signature σ_{ID}, the verification algorithm outputs `valid` or $\perp$ (indicating invalid).

Representative PQ signature schemes include ML-DSA (CRYSTALS-Dilithium) [5, 61], SLH-DSA [16, 62], FALCON [6], HBS [17], HAWK [25], and HAETAE [22].

3 Another Hybrid EDHOC Protocol

In this section, we propose another hybrid EDHOC (for short, Hybrid-EDHOC2) protocol that provides both classical security and PQ security by instantiating KEM (resp., signature scheme) with PQ KEM (resp., PQ signature scheme). Specifically, the Hybrid-EDHOC2 protocol is based on an authentication method of static ECDH keys and static PQ signature keys. Main ideas of the Hybrid-EDHOC2 protocol are that (1) static ECDH raw public keys and static PQ signature raw verification keys are dealt with pre-shared secrets between initiator I and responder R; (2) an ephemeral PQ KEM is used for forward secrecy; (3) a PQ KEM shared key is used as an additional key material in the computation of PRK; and (4) static PQ signature key is used for PQ security against key compromise impersonation. In the Hybrid-EDHOC2 protocol, initiator I and responder R share an authenticated session key PRK_{out}. Also, the Hybrid-EDHOC2 protocol provides forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation. Before executing the Hybrid-EDHOC2 protocol, I holds $((a, A \equiv g^a), (sk_I, VK_I), (R, B, VK_R))$ secretly and R stores $((b, B \equiv g^b), (sk_R, VK_R), (I, A, VK_I))$ secretly.[1] For a graphical description of the Hybrid-EDHOC2 protocol, see Figure 1.

[1] These pre-shared secrets are installed or provisioned out-of-band in the initialization phase (e.g., at the manufacturing process of IoT devices). See Section 5.1 for a more detailed discussion.

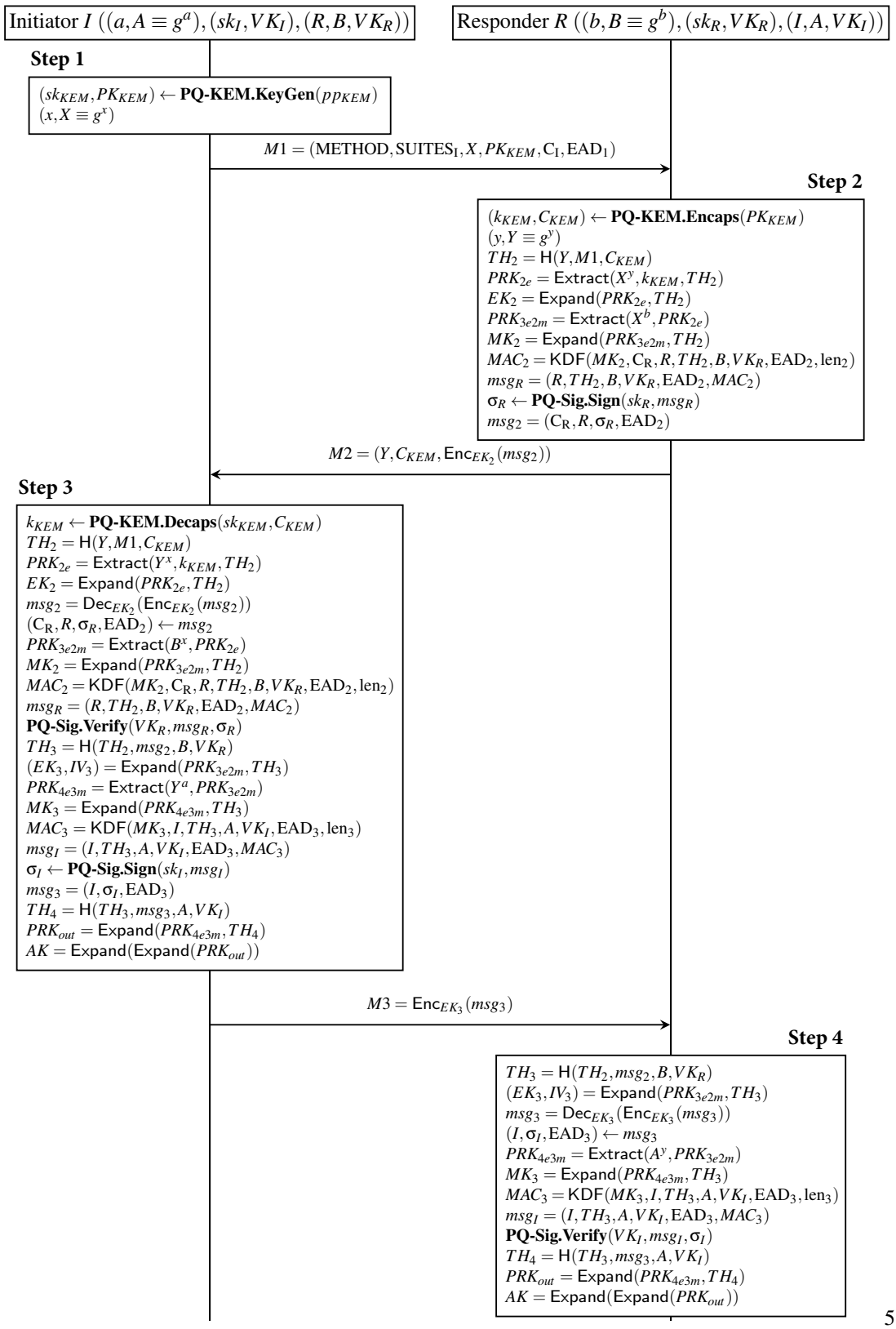


Figure 1: The Hybrid-EDHOC2 protocol

3.1 Step 1 (Initiator)

Initiator I executes the followings:

1. (Pre-computable) Generate an ephemeral KEM private-public key pair $(sk_{KEM}, PK_{KEM}) \leftarrow$ **PQ-KEM.KeyGen**(pp_{KEM})
2. (Pre-computable) Generate an ephemeral ECDH private-public key pair (x, X) such that $X \equiv g^x$
3. Set $M1 = (\text{METHOD}, \text{SUITES}_\text{I}, X, PK_{KEM}, \text{C}_\text{I}, \text{EAD}_1)$

I sends $M1$ to R.

3.2 Step 2 (Responder)

Upon receiving $M1$ from I, responder R executes the followings:

1. Compute a KEM shared key and a KEM ciphertext $(k_{KEM}, C_{KEM}) \leftarrow$ **PQ-KEM.Encaps**(PK_{KEM})
2. (Pre-computable) Generate an ephemeral ECDH private-public key pair (y, Y) such that $Y \equiv g^y$
3. Compute $TH_2 = \mathsf{H}(Y, M1, C_{KEM})$
4. Compute $PRK_{2e} = \mathsf{Extract}(X^y, k_{KEM}, TH_2)$ and $EK_2 = \mathsf{Expand}(PRK_{2e}, TH_2)$
5. Compute $PRK_{3e2m} = \mathsf{Extract}(X^b, PRK_{2e})$ and $MK_2 = \mathsf{Expand}(PRK_{3e2m}, TH_2)$
6. Compute $MAC_2 = \mathsf{KDF}(MK_2, \text{C}_\text{R}, R, TH_2, B, VK_R, \text{EAD}_2, \text{len}_2)$
7. Set $msg_R = (R, TH_2, B, VK_R, \text{EAD}_2, MAC_2)$
8. Compute a signature $\sigma_R \leftarrow$ **PQ-Sig.Sign**(sk_R, msg_R)
9. Set $msg_2 = (\text{C}_\text{R}, R, \sigma_R, \text{EAD}_2)$
10. Set $M2 = (Y, C_{KEM}, \mathsf{Enc}_{EK_2}(msg_2))$

R sends $M2$ to I.

3.3 Step 3 (Initiator)

Upon receiving $M2$ from R, initiator I executes the followings:

1. Compute a KEM shared key $k_{KEM} \leftarrow$ **PQ-KEM.Decaps**(sk_{KEM}, C_{KEM})
2. Compute $TH_2 = \mathsf{H}(Y, M1, C_{KEM})$
3. Compute $PRK_{2e} = \mathsf{Extract}(Y^x, k_{KEM}, TH_2)$ and $EK_2 = \mathsf{Expand}(PRK_{2e}, TH_2)$
4. Compute $msg_2 = \mathsf{Dec}_{EK_2}(\mathsf{Enc}_{EK_2}(msg_2))$
5. Parse $(\text{C}_\text{R}, R, \sigma_R, \text{EAD}_2) \leftarrow msg_2$
6. Compute $PRK_{3e2m} = \mathsf{Extract}(B^x, PRK_{2e})$ and $MK_2 = \mathsf{Expand}(PRK_{3e2m}, TH_2)$
7. Compute $MAC_2 = \mathsf{KDF}(MK_2, \text{C}_\text{R}, R, TH_2, B, VK_R, \text{EAD}_2, \text{len}_2)$

8. Set $msg_R = (R, TH_2, B, VK_R, \mathrm{EAD}_2, MAC_2)$
9. If **PQ-Sig.Verify**(VK_R, msg_R, σ_R) is `valid`, proceed to the next.
10. Compute $TH_3 = \mathsf{H}(TH_2, msg_2, B, VK_R)$
11. Compute $(EK_3, IV_3) = \mathsf{Expand}(PRK_{3e2m}, TH_3)$
12. Compute $PRK_{4e3m} = \mathsf{Extract}(Y^a, PRK_{3e2m})$ and $MK_3 = \mathsf{Expand}(PRK_{4e3m}, TH_3)$
13. Compute $MAC_3 = \mathsf{KDF}(MK_3, I, TH_3, A, VK_I, \mathrm{EAD}_3, \mathrm{len}_3)$
14. Set $msg_I = (I, TH_3, A, VK_I, \mathrm{EAD}_3, MAC_3)$
15. Compute a signature $\sigma_I \leftarrow$ **PQ-Sig.Sign**(sk_I, msg_I)
16. Set $msg_3 = (I, \sigma_I, \mathrm{EAD}_3)$
17. Set $M3 = \mathsf{Enc}_{EK_3}(msg_3)$
18. Compute $TH_4 = \mathsf{H}(TH_3, msg_3, A, VK_I)$
19. Compute a session key $PRK_{out} = \mathsf{Expand}(PRK_{4e3m}, TH_4)$ and an application key $AK = \mathsf{Expand}(\mathsf{Expand}(PRK_{out}))$

I sends $M3$ to R.

3.4 Step 4 (Responder)

Upon receiving $M3$ from I, responder R executes the followings:

1. Compute $TH_3 = \mathsf{H}(TH_2, msg_2, B, VK_R)$
2. Compute $(EK_3, IV_3) = \mathsf{Expand}(PRK_{3e2m}, TH_3)$
3. Compute $msg_3 = \mathsf{Dec}_{EK_3}(\mathsf{Enc}_{EK_3}(msg_3))$
4. Parse $(I, \sigma_I, \mathrm{EAD}_3) \leftarrow msg_3$
5. Compute $PRK_{4e3m} = \mathsf{Extract}(A^y, PRK_{3e2m})$ and $MK_3 = \mathsf{Expand}(PRK_{4e3m}, TH_3)$
6. Compute $MAC_3 = \mathsf{KDF}(MK_3, I, TH_3, A, VK_I, \mathrm{EAD}_3, \mathrm{len}_3)$
7. Set $msg_I = (I, TH_3, A, VK_I, \mathrm{EAD}_3, MAC_3)$
8. If **PQ-Sig.Verify**(VK_I, msg_I, σ_I) is `valid`, proceed to the next.
9. Compute $TH_4 = \mathsf{H}(TH_3, msg_3, A, VK_I)$
10. Compute a session key $PRK_{out} = \mathsf{Expand}(PRK_{4e3m}, TH_4)$ and an application key $AK = \mathsf{Expand}(\mathsf{Expand}(PRK_{out}))$

In the above, Enc in $M2$ is an XOR stream cipher, and Enc in $M3$ is an AEAD (Authenticated Encryption with Associated Data). Also, an optional fourth message for key confirmation is omitted as in the EDHOC protocol [52].

Note that the Hybrid-EDHOC protocol [66] inherently does not provide PQ security against key compromise impersonation since its construction is based on an authentication method of static ECDH keys only. For example, a quantum attacker, who gets responder R's secrets $((b,B),(I,A))$, can impersonate initiator I by retrieving the static ECDH private key a from the static ECDH raw public key A.

4 Security

Here, we discuss several classical and PQ security properties (mutual authentication, security of session key, forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation) of the Hybrid-EDHOC2 protocol.

4.1 Mutual Authentication and Security of Session Key

In general, (semantic) security of session key implies mutual authentication in authenticated key exchange protocols. Suppose a classical or quantum attacker, who can fully control the communications by eavesdropping/inserting/modifying messages and impersonating one peer. The Hybrid-EDHOC2 protocol provides mutual authentication and security of session key since static ECDH keys and static PQ signature keys are used for authentication between initiator I and responder R. Regarding classical security of session key, a classical attacker cannot compute an authenticated ephemeral-static ECDH key g^{xb} and an authenticated static-ephemeral ECDH key g^{ay}, both of which are used in the computation of PRK. The same reason applies to PQ security of session key since the static ECDH raw public keys (g^a and g^b) are pre-shared secrets as well, and these public keys do not appear in public.

4.2 Forward Secrecy

Suppose a classical or quantum attacker, who gets all the secrets $((a,A),(sk_I,VK_I)),((b,B),(sk_R,VK_R))$ from initiator I and responder R, and tries to find out the previously-established session key PRK_{out}. The Hybrid-EDHOC2 protocol provides forward secrecy since ephemeral ECDH keys and an ephemeral PQ KEM are used. Regarding classical forward secrecy, a classical attacker cannot compute an ephemeral-ephemeral ECDH key g^{xy} used in the key derivation schedule of PRK_{out}. Regarding PQ forward secrecy, a quantum attacker cannot compute an ephemeral PQ KEM shard key k_{KEM} used in the key derivation schedule of PRK_{out}.

4.3 Identity Protection of I

Suppose a classical or quantum attacker, who can fully control the communications by eavesdropping/inserting/modifying messages and impersonating responder R, and tries to find out initiator identity I. As in the EDHOC protocol [52], the Hybrid-EDHOC2 protocol provides identity protection of I since initiator identity I is sent in an encrypted form $M3$ with an encryption key EK_3, and only the legitimate responder R can decrypt the third message $M3$. Regarding classical identity protection of I, a classical attacker cannot compute an authenticated ephemeral-static ECDH key g^{xb}, used in the computation of EK_3 and a MAC value MAC_2. Regarding PQ identity protection of I, a quantum attacker cannot compute an authenticated ephemeral-static ECDH key g^{xb}, used in the computation of EK_3 and a MAC value MAC_2, and cannot generate a valid PQ signature σ_R.

4.4 Security against Identity Misbinding

Suppose a classical or quantum attacker, who can fully control the communications by eavesdropping/inserting/modifying messages and impersonating one peer, and then tries to make a third peer share a session key PRK_{out} without the initiator I and responder R noticing. The Hybrid-EDHOC2 protocol provides security against identity misbinding since initiator identity I and responder identity R are authenticated by using MACs and PQ signatures, and these identities are included in the key derivation schedule of PRK_{out}. Regarding classical security against identity misbinding, a classical attacker cannot compute

an authenticated ephemeral-static ECDH key g^{xb}, used in the computation of MAC_2, and an authenticated static-ephemeral ECDH key g^{ay}, used in the computation of MAC_3. Regarding PQ security against identity misbinding, a quantum attacker cannot compute an authenticated ephemeral-static ECDH key g^{xb}, used in the computation of MAC_2, and an authenticated static-ephemeral ECDH key g^{ay}, used in the computation of MAC_3, and cannot generate valid PQ signatures σ_R and σ_I.

4.5 Security against Key Compromise Impersonation

Suppose a classical or quantum attacker, who gets initiator I's secrets $((a,A),(sk_I,VK_I),(R,B,VK_R))$ (resp., responder R's secrets $((b,B),(sk_R,VK_R),(I,A,VK_I))$, and then tries to impersonate responder R (resp., initiator I). The Hybrid-EDHOC2 protocol provides security against key compromise impersonation since the static ECDH private keys and PQ signatures are used in both initiator I and responder R sides. Regarding classical security against key compromise impersonation, a classical attacker, who gets initiator I's secrets (resp., responder R's secrets) and impersonates responder R (resp., initiator I), cannot compute an authenticated ephemeral-static ECDH key g^{xb} (resp., an authenticated static-ephemeral ECDH key g^{ay}), used in the computation of MAC_2 (resp., MAC_3). Regarding PQ security against key compromise impersonation, we give a reduction to EUF-CMA (Existential Unforgeability under Chosen Message Attack) security of a PQ signature scheme. Let $\mathcal{B}$ be a quantum attacker who can break the EUF-CMA security of the PQ signature scheme. We assume that there exists a quantum attacker $\mathcal{A}$ who can break PQ security against key compromise impersonation in the Hybrid-EDHOC2 protocol. Whenever a PQ signing query comes from $\mathcal{A}$, $\mathcal{B}$ forwards the query to its own PQ signing oracle. By using $\mathcal{A}$, we can trivially construct the attacker $\mathcal{B}$ since the only winning way for $\mathcal{A}$ is to generate a valid PQ signature without knowing the static PQ signature signing key sk. Of course, the static PQ signature verification key VK is useless to $\mathcal{A}$.

5 Comparison

In this section, we compare the EDHOC [52] and relevant protocols (PQ-EDHOC [31], PQ-EDHOC1 [30], PQ-EDHOC2 [30], Hybrid-EDHOC [66], and Hybrid-EDHOC2 (Section 3)) with respect to several classical and PQ security properties and efficiency in terms of computation costs and communication bandwidths.

5.1 Classification

Before security and efficiency comparisons, we first classify the EDHOC [52], Hybrid-EDHOC2 (Section 3), and relevant protocols [31, 30, 66, 40] in Table 2, according to whether each protocol depends on PKI (Public Key Infrastructure) or not.

Table 2: Classification of the EDHOC [52], Hybrid-EDHOC2, and relevant protocols

	PKI-based (Scalable)	Secret-key based (Not scalable)
Classical security	EDHOC (Method Type 0, 1, 2, 3) [52]	EDHOC-PSK [40]
PQ security	PQ-EDHOC [31], PQ-EDHOC1 [30], PQ-EDHOC2 [30]	EDHOC-PSK [40]
Hybrid security		Hybrid-EDHOC [66], Hybrid-EDHOC2 (Section 3)

A distinctive feature of the Hybrid-EDHOC2 protocol is that public-key certificates for initiator I and responder R are not used, and static ECDH raw public keys and static PQ signature raw verification keys are treated as pre-shared secrets. In practice, the initiator I (resp., the responder R) is an IoT device (resp., a backend IoT platform/server), and most IoT devices are configured with pre-shared secrets installed or provisioned out-of-band at the manufacturing process for secure communication (e.g., firmware updates). Of course, these secrets are shared with (and managed by) the corresponding backend IoT platforms/servers. Other examples include AKMA (Authentication and Key Management for Applications) [2] and EAP (Extensible Authentication Protocol) [47, 46]. In this regard, the Hybrid-EDHOC2 protocol is more suitable for the real world than the EDHOC [52], PQ-EDHOC [31], PQ-EDHOC1 [30], and PQ-EDHOC2 [30] protocols. In addition, the Hybrid-EDHOC2 protocol can reduce the computation, memory, communication bandwidth, and energy costs of validating public-key certificates. This is a reason why the IETF LAKE working group has been standardizing a pre-shared key authentication method for EDHOC [40]. However, the EDHOC-PSK protocol [40] is an Internet-Draft and a work in progress.

Table 3 shows a design difference between the Hybrid-EDHOC [66] and Hybrid-EDHOC2 protocols, where the latter provides double authentication based on a MAC and a PQ signature.

Table 3: Design difference between the Hybrid-EDHOC [66] and Hybrid-EDHOC2 protocols

	Hybrid-EDHOC [66]	Hybrid-EDHOC2 (Section 3)
Static ECDH key pair	Yes	Yes
Static PQ signature key pair	No	Yes
MAC authentication	Yes	Yes (Implicit)
Signature authentication	No	Yes

5.2 Security and Efficiency Comparisons

In Table 4, we compare the EDHOC [52] and relevant protocols with respect to several classical and PQ security properties (mutual authentication, security of session key, forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation). It is clear that the Hybrid-EDHOC2 protocol provides both classical security and PQ security simultaneously.

In Tables 5, 6, 7, and 8, we compare the EDHOC [52] with relevant protocols with respect to computation and communication efficiency. Note that the PQ-EDHOC1 [30] and PQ-EDHOC2 [30] protocols require substantial modifications to the original EDHOC protocol [52, 53]. Specifically, the PQ-EDHOC1 protocol needs additional fourth and fifth messages to be exchanged, and the PQ-EDHOC2 protocol needs to send a KEM ciphertext, computed with the responder's KEM public key, in the first message. Compared to PQ-EDHOC1 [30] and PQ-EDHOC2 [30], the Hybrid-EDHOC2 protocol does not alter the core EDHOC message structure at all.

6 Conclusion

In this paper, we proposed another hybrid EDHOC (Hybrid-EDHOC2) protocol that provides both classical security and PQ security by instantiating KEM (resp., signature scheme) with PQ KEM (resp., PQ signature scheme). Specifically, the Hybrid-EDHOC2 protocol is based on an authentication method of static ECDH keys and static PQ signature keys. In the Hybrid-EDHOC2 protocol, initiator I and responder R share an authenticated session key PRK_{out}. Unlike Hybrid-EDHOC [66], the Hybrid-EDHOC2

Table 4: Security comparison of the EDHOC [52] and relevant protocols

	EDHOC [52]		PQ-EDHOC [31], PQ-EDHOC1 [30], PQ-EDHOC2 [30]		Hybrid-EDHOC [66]		Hybrid-EDHOC2 (Section 3)	
	Classical security	PQ security	Classical security	PQ security	Classical security	PQ security	Classical security	PQ security
Mutual authentication	Yes	No	No	Yes	Yes	Yes	Yes	Yes
Security of session key	Yes	No	No	Yes	Yes	Yes	Yes	Yes
Forward secrecy	Yes	No	No	Yes	Yes	Yes	Yes	Yes
Identity protection of I	Yes	No	No	Yes	Yes	Yes	Yes	Yes
Security against identity misbinding	Yes	No	No	Yes	Yes	Yes	Yes	Yes
Security against key compromise impersonation	Yes	No	No	Yes	Yes	No	Yes	Yes

Table 5: Computation efficiency comparison of the EDHOC [52], Hybrid-EDHOC [66], and Hybrid-EDHOC2 protocols, where the numbers in parentheses are computation costs excluding pre-computable ones

	EDHOC (Method Type 3) [52]		EDHOC (Method Type 0) [52]		Hybrid-EDHOC [66]		Hybrid-EDHOC2 (Section 3)	
	I	R	I	R	I	R	I	R
Scalar multiplication	4 (3)	4 (3)	2 (1)	2 (1)	4 (3)	4 (3)	4 (3)	4 (3)
Sig.Sign			1	1				
Sig.Verify			1	1				
PQ-KEM.KeyGen					1 (0)		1 (0)	
PQ-KEM.Encaps						1		1
PQ-KEM.Decaps					1		1	
PQ-Sig.Sign							1	1
PQ-Sig.Verify							1	1
Certificate validation	1	1	1	1				

protocol provides double authentication based on a MAC and a PQ signature, and PQ security against key compromise impersonation. Also, we discussed several classical and PQ security properties (mutual authentication, security of session key, forward secrecy, identity protection, security against identity misbinding, and security against key compromise impersonation) of the Hybrid-EDHOC2 protocol. In

Table 6: Computation efficiency comparison of the PQ-EDHOC [31], PQ-EDHOC1 [30], and PQ-EDHOC2 [30] protocols, where the numbers in parentheses are computation costs excluding pre-computable ones

	PQ-EDHOC [31]		PQ-EDHOC1 [30]		PQ-EDHOC2 [30]	
	I	*R*	*I*	*R*	*I*	*R*
PQ-KEM.KeyGen	1 (0)		1 (0)		1 (0)	
PQ-KEM.Encaps		1	1	2	1	2
PQ-KEM.Decaps	1		2	1	2	1
PQ-Sig.Sign	1	1				
PQ-Sig.Verify	1	1				
PQ certificate validation	1	1	1	1		1

Table 7: Communication efficiency comparison of the EDHOC [52], Hybrid-EDHOC [66], and Hybrid-EDHOC2 protocols, where $\Delta = |\text{METHOD}| + |\text{SUITES}_\text{I}| + |X| + |\text{C}_\text{I}| + |\text{EAD}_1|$ and $|\cdot|$ indicates a bit-length

	EDHOC (Method Type 3) [52]	EDHOC (Method Type 0) [52]	Hybrid-EDHOC [66]	Hybrid-EDHOC2 (Section 3)
$M1$	Δ	Δ	$\Delta + \|PK_{KEM}\|$	$\Delta + \|PK_{KEM}\|$
$M2$	$\|Y\| + \|\text{Enc}\|$	$\|Y\| + \|\text{Enc}\|$	$\|Y\| + \|\text{Enc}\| + \|C_{KEM}\|$	$\|Y\| + \|\text{Enc}\| + \|C_{KEM}\|$
$M3$	$\|\text{Enc}\|$	$\|\text{Enc}\|$	$\|\text{Enc}\|$	$\|\text{Enc}\|$

addition, we conducted extensive comparisons of the EDHOC [52], PQ-EDHOC [31], PQ-EDHOC1 [30], PQ-EDHOC2 [30], Hybrid-EDHOC [66], and Hybrid-EDHOC2 (Section 3) protocols with respect to security properties and computation/communication efficiency.

Future works include (1) formal proofs of the Hybrid-EDHOC2 protocol by using an automated symbolic security verification tool (e.g., ProVerif [15] or Tamarin Prover [18]); (2) implementation of the Hybrid-EDHOC2 protocol with a concrete PQ KEM (e.g., ML-KEM) and a concrete PQ signature scheme (e.g., ML-DSA) for NIST PQC security strength categories [14]; 3) quantitative performance measurements (e.g., runtime, memory usage, message size, and energy cost) and evaluation of the Hybrid-EDHOC2 protocol on constrained IoT devices; and (4) handling network-layer fragmentation and packet-loss overhead during the Hybrid-EDHOC2 handshake, while considering the current IEEE 802.15.4 [8] or 6LoWPAN [9] packet sizes.

Acknowledgments

We sincerely appreciate the anonymous reviewers' constructive and valuable comments on this paper.

Table 8: Communication efficiency comparison of the PQ-EDHOC [31], PQ-EDHOC1 [30], and PQ-EDHOC2 [30] protocols, where $\delta = |\text{METHOD}| + |\text{SUITES}_\text{I}| + |\text{C}_\text{I}| + |\text{EAD}_1|$ and $|\cdot|$ indicates a bit-length

	PQ-EDHOC [31]	PQ-EDHOC1 [30]	PQ-EDHOC2 [30]
$M1$	$\delta + \lvert PK_{KEM}\rvert$	$\delta + \lvert PK_{KEM}\rvert$	$\lvert PK_{KEM}\rvert + \lvert C_{KEM}\rvert + \lvert\text{Enc}\rvert$
$M2$	$\lvert\text{Enc}\rvert + \lvert C_{KEM}\rvert$	$\lvert\text{Enc}\rvert + \lvert C_{KEM}\rvert$	$\lvert\text{Enc}\rvert + 2\lvert C_{KEM}\rvert$
$M3$	$\lvert\text{Enc}\rvert$	$\lvert\text{Enc}\rvert + \lvert C_{KEM}\rvert$	$\lvert\text{Enc}\rvert$
$M4$		$\lvert\text{Enc}\rvert + \lvert C_{KEM}\rvert$	
$M5$		$\lvert\text{Enc}\rvert$	

References

[1] ANSSI views on the Post-Quantum Cryptography transition. https://messervices.cyber.gouv.fr/guides/en-anssi-views-post-quantum-cryptography-transition.

[2] Authentication and Key Management for Applications (AKMA) in 5G. https://www.3gpp.org/technologies/akma.

[3] BIKE - Bit Flipping Key Encapsulation. https://bikesuite.org/.

[4] Classic McEliece. https://classic.mceliece.org/.

[5] CRYSTALS. https://pq-crystals.org/.

[6] FALCON. https://falcon-sign.info/.

[7] FrodoKEM. https://frodokem.org/.

[8] IEEE Standard for Low-Rate Wireless Networks. https://standards.ieee.org/ieee/802.15.4/11041/.

[9] IPv6 over Low power WPAN (6lowpan). https://datatracker.ietf.org/wg/6lowpan/about/.

[10] Lightweight Authenticated Key Exchange (lake). https://datatracker.ietf.org/group/lake/about/.

[11] Migration to Post-Quantum Cryptography. https://www.nccoe.nist.gov/crypto-agility-considerations-migrating-post-quantum-cryptographic-algorithms.

[12] Noise Protocol Framework. https://noiseprotocol.org/.

[13] Post-Quantum Cryptography. https://csrc.nist.gov/projects/post-quantum-cryptography.

[14] Post-Quantum Cryptography, Security (Evaluation Criteria). https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/evaluation-criteria/security-(evaluation-criteria).

[15] ProVerif. https://bblanche.gitlabpages.inria.fr/proverif/.

[16] SPHINCS+. https://sphincs.org/.

[17] Stateful Hash-Based Signatures. https://csrc.nist.gov/projects/stateful-hash-based-signatures.

[18] The Tamarin Prover. https://tamarin-prover.com/.

[19] Y. Angel, B. Dowling, A. Hülsing, P. Schwabe, and F. Weber. Post Quantum Noise. In *CCS 2022*, pages 97–109. ACM, 2022.

[20] N. Bindel, J. Brendel, M. Fischlin, B. Goncalves, and D. Stebila. Hybrid Key Encapsulation Mechanisms and Authenticated Key Exchange. In *PQCrypto 2019*, pages 206–226. Springer, 2019.

[21] BSI TR-02102-1. Cryptographic Mechanisms: Recommendations and Key Lengths. https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html, January 2024.

[22] J. H. Cheon, H. Choe, J. Devevey, T. Güneysu, D. Hong, M. Krausz, G. Land, M. Möller, D. Stehlé, and M. Yi. HAETAE: Shorter Lattice-Based Fiat-Shamir Signatures. *IACR Transactions on Cryptographic Hardware and Embedded Systems*, 2024:25–75, 2024.

[23] B. Cottier and D. Pointcheval. Security Analysis of Improved EDHOC Protocol. In *Foundations and Practice of Security (FPS) 2022*, pages 3–18. Springer, 2022.

[24] C. Cremers, A. Dax, and N. Medinger. Keeping Up with the KEMs: Stronger Security Notions for KEMs and Automated Analysis of KEM-based Protocols. In *CCS 2024*, pages 1046–1060. ACM, 2024.

[25] L. Ducas, E. W. Postlethwaite, L. N. Pulles, and W. v. Woerden. Hawk: Module LIP Makes Lattice Signatures Fast, Compact and Simple. In *ASIACRYPT 2022*, pages 65–94. Springer, 2022.

[26] ETSI TS 103 744. CYBER; Quantum-safe Hybrid Key Exchanges. https://www.etsi.org/deliver/etsi_ts/103700_103799/103744/01.01.01_60/ts_103744v010101p.pdf, December 2020.

[27] G. Fedrecheski, M. Vučinić, and T. Watteyne. Performance Comparison of EDHOC and DTLS 1.3 in Internet-of-Things Environments. In *Wireless Communications and Networking Conference (WCNC) 2024*. IEEE, 2024.

[28] L. Ferreira. Computational Security Analysis of the Full EDHOC Protocol. In *CT-RSA 2024*, pages 25–48. Springer, 2024.

[29] R. Fiedler and F. Günther. Security Analysis of Signal's PQXDH Handshake. In *PKC 2025*, pages 137–169. Springer, 2025.

[30] L. P. Fraile, C. Koulamas, and A. P. Fournaris. Reinventing EDHOC for the Post-Quantum Era. *IEEE Access*, 13:196622–196640, 2025.

[31] L. P. Fraile, G. Tasopoulos, C. Koulamas, R. K. Zhao, N. H. Sultan, F. Regazzoni, and A. P. Fournaris. Enabling Quantum-Resistant EDHOC: Design and Performance Evaluation. *IEEE Access*, 13:75861–75884, 2025.

[32] F. Günther and M. I. T. Mukendi. Careful with MAc-then-SIGn: A Computational Analysis of the EDHOC Lightweight Authenticated Key Exchange Protocol. In *EuroS&P 2023*, pages 773–796. IEEE, 2023.

[33] IETF Internet-Draft. Amortized PQ MLS Combiner. https://www.ietf.org/archive/id/draft-ietf-mls-combiner-02.html, October 2025.

[34] IETF Internet-Draft. Hybrid key exchange in TLS 1.3. https://www.ietf.org/archive/id/draft-ietf-tls-hybrid-design-16.html, September 2025.

[35] IETF Internet-Draft. Hybrid signature spectrums. https://www.ietf.org/archive/id/draft-ietf-pquip-hybrid-signature-spectrums-07.html, June 2025.

[36] IETF Internet-Draft. Post-Quantum Cryptography for Engineers. https://www.ietf.org/archive/id/draft-ietf-pquip-pqc-engineers-14.html, August 2025.

[37] IETF Internet-Draft. Composite ML-KEM for use in X.509 Public Key Infrastructure. https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-kem-14.html, March 2026.

[38] IETF Internet-Draft. Composite Module-Lattice-Based Digital Signature Algorithm (ML-DSA) for use in X.509 Public Key Infrastructure. https://www.ietf.org/archive/id/draft-ietf-lamps-pq-composite-sigs-18.html, April 2026.

[39] IETF Internet-Draft. Concrete Hybrid PQ/T Key Encapsulation Mechanisms. https://www.ietf.org/archive/id/draft-irtf-cfrg-concrete-hybrid-kems-03.html, March 2026.

[40] IETF Internet-Draft. EDHOC Authenticated with Pre-Shared Keys (PSK). https://www.ietf.org/archive/id/draft-ietf-lake-edhoc-psk-07.html, March 2026.

[41] IETF Internet-Draft. Ephemeral Diffie-Hellman Over COSE (EDHOC) and Object Security for Constrained Environments (OSCORE) Profile for Authentication and Authorization for Constrained Environments (ACE). https://www.ietf.org/archive/id/draft-ietf-ace-edhoc-oscore-profile-10.html, March 2026.

[42] IETF Internet-Draft. Hybrid PQ/T Key Encapsulation Mechanisms. https://www.ietf.org/archive/id/draft-irtf-cfrg-hybrid-kems-10.html, March 2026.

[43] IETF Internet-Draft. PQ/T Composite Schemes for OpenPGP using NIST and Brainpool Elliptic Curve

Domain Parameters. https://www.ietf.org/archive/id/draft-ietf-openpgp-nist-bp-comp-03.html, January 2026.

[44] IETF Internet-Draft. PQ/T Hybrid Composite Signatures for JOSE and COSE. https://www.ietf.org/archive/id/draft-ietf-jose-pq-composite-sigs-01.html, February 2026.

[45] IETF Internet-Draft. Remote attestation over EDHOC. https://www.ietf.org/archive/id/draft-ietf-lake-ra-04.html, March 2026.

[46] IETF Internet-Draft. Using the Extensible Authentication Protocol (EAP) with Ephemeral Diffie-Hellman over COSE (EDHOC). https://www.ietf.org/archive/id/draft-ietf-emu-eap-edhoc-08.html, March 2026.

[47] IETF RFC 3748. Extensible Authentication Protocol (EAP). https://www.rfc-editor.org/rfc/rfc3748.html, June 2004.

[48] IETF RFC 7296. Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc7296.html, October 2014.

[49] IETF RFC 8446. The Transport Layer Security (TLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc8446.html, August 2018.

[50] IETF RFC 9147. The Datagram Transport Layer Security (DTLS) Protocol Version 1.3. https://www.rfc-editor.org/rfc/rfc9147.html, April 2022.

[51] IETF RFC 9370. Multiple Key Exchanges in the Internet Key Exchange Protocol Version 2 (IKEv2). https://www.rfc-editor.org/rfc/rfc9370.html, May 2023.

[52] IETF RFC 9528. Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9528.html, March 2024.

[53] IETF RFC 9529. Traces of Ephemeral Diffie-Hellman Over COSE (EDHOC). https://www.rfc-editor.org/rfc/rfc9529.html, March 2024.

[54] IETF RFC 9794. Terminology for Post-Quantum Traditional Hybrid Schemes. https://www.rfc-editor.org/rfc/rfc9794.html, June 2025.

[55] ISO/IEC 18033-2:2006/Amd 2. Information technology — Security techniques — Encryption algorithms — Part 2: Asymmetric ciphers Amendment 2 (Under development). https://www.iso.org/standard/86890.html.

[56] C. Jacomme, E. Klein, S. Kremer, and M. Racouchot. A comprehensive, formal and automated analysis of the EDHOC protocol. In *USENIX Security Symposium 2023*, pages 5881–5898. USENIX Association, 2023.

[57] J. Katz and Y. Lindell. *Introduction to Modern Cryptography*. Chapman and Hall/CRC, 2025.

[58] J. Kim, D. G. Duguma, S. Lee, B. Kim, J. Lim, and I. You. Scrutinizing the Vulnerability of Ephemeral Diffie–Hellman over COSE (EDHOC) for IoT Environment Using Formal Approaches. *Mobile Information Systems*, 2021, 2021.

[59] H. Krawczyk. SIGMA: The 'SIGn-and-MAc' Approach to Authenticated Diffie-Hellman and Its Use in the IKE Protocols. In *CRYPTO 2003*, pages 400–425. Springer, 2003.

[60] NIST FIPS 203. Module-Lattice-Based Key-Encapsulation Mechanism Standard. https://csrc.nist.gov/pubs/fips/203/final, August 2024.

[61] NIST FIPS 204. Module-Lattice-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/204/final, August 2024.

[62] NIST FIPS 205. Stateless Hash-Based Digital Signature Standard. https://csrc.nist.gov/pubs/fips/205/final, August 2024.

[63] NIST SP 800-227. Recommendations for Key-Encapsulation Mechanisms. https://csrc.nist.gov/pubs/sp/800/227/final, September 2025.

[64] K. Norrman, V. Sundararajan, and A. Bruni. Formal Analysis of EDHOC Key Establishment for Constrained IoT Devices. In *SECRYPT 2021*, pages 210–221. SCITEPRESS, 2021.

[65] E. L. Pérez, G. Selander, J. P. Mattsson, T. Watteyne, and M. Vučinić. EDHOC is a New Security Handshake Standard: An Overview of Security Analysis. *Computer*, 57(9):101–110, 2024.

[66] S. H. Shin, A. P. S. Iskandar, Y. Ko, and I. You. A Hybrid EDHOC Protocol. In *The 9th International Conference on Mobile Internet Security (MobiSec 2025)*, 2025.

[67] P. W. Shor. Algorithms for quantum computation: discrete logarithms and factoring. In *The 35th Annual Symposium on Foundations of Computer Science*, pages 124–134. IEEE, 1994.